

深圳大学实验报告

课程名称： 互联网编程

实验项目名称： 域名 IP 转换及应用 URL 类定位和获取数据编程

学院： 计算机与软件学院

专业： 计算机科学与技术

指导教师： 李梦柯

报告人： 姚泽棉 学号： 2024150026 班级： 高性能班

实验时间： 2026/05/14

实验报告提交时间： 2026/05/21

教务处制

一、实验目的与内容：

目的：

- 1.掌握网络、文件 IO 流相关编程技术及能力。
- 2.了解域名系统（DNS）原理，掌握 `InetAddress` 类及 `Inet4Address/Inet6Address` 的使用方法。
- 3 熟悉 `NetworkInterface` 类的使用方法。
- 4.掌握 `URL` 类的使用，在不考虑所使用协议的细节情况下，会用 java 编写程序在网络上定位和获取数据。

内容要求：

1. 编写域名解析程序（30 分）：
编写一个可重用的域名解析程序模块，使之能够将用户输入的域名解析为 IP 地址。
2. 编写下载功能程序（40 分）：
使用 `URL` 类，编写一个可重用的下载程序模块，完成 HTML 和图像文件的下载。即，用户输入一个 `URL`，程序会将 `URL` 所指向的网络资源下载到本地，并保存到**指定**磁盘位置。
3. 编写断点续传下载功能程序（30 分）：
选择网络上一个较大的文件（如 10MB 以上），并继续完善 2 中的文件下载程序，使其具有“断点续传”功能，改写成为一个可重用的具“断点续传”功能的可重用程序模块。

注意：

1. 实验报告中需要有实验结果的截图。
2. 可能会在实验课随机抽查代码理解情况。

二、实验过程和代码与结果

- 1.给出满足内容要求 1 的程序源码及运行结果，**简述思路或实验过程**。

简述思路

本实验通过 Java 提供的 `InetAddress` 类实现域名解析功能。程序接收用户输入的域名，调用 `InetAddress.getAllByName()` 方法向 DNS 服务器查询对应的 IP 地址，并返回结果列表。随后对返回的地址进行遍历输出，从而实现域名到 IP 地址的映射。

程序源码

```

1  import java.net.InetAddress;
2
3  public class DNSResolver {
4      public static void resolve(String domain) { 1个用法
5          try {
6              InetAddress[] addresses = InetAddress.getAllByName(domain);
7
8              System.out.println("域名: " + domain);
9              System.out.println("解析结果: ");
10
11             for (InetAddress addr : addresses) {
12                 System.out.println(addr.getHostAddress());
13             }
14
15         } catch (Exception e) {
16             System.out.println("解析失败: " + e.getMessage());
17         }
18     }
19
20     public static void main(String[] args) {
21         resolve(domain: "www.baidu.com");
22     }
23 }

```

运行结果示例

域名: www.baidu.com

解析结果:

111.45.11.5

183.240.99.224

2409:8c54:870:310:0:ff:b0ed:40ac

2409:8c54:870:187:0:ff:b0d9:bb1c

进程已结束, 退出代码为 0

问题和解决方法:

在实验过程中, 了解到域名解析依赖系统当前配置的 DNS 服务器, 因此解析结果可能会受到网络环境 (如校园网、代理服务器等) 的影响。关闭 VPN 即可。

2. 给出满足内容要求 2 的程序源码及运行结果, 简述思路或实验过程。

简述思路

本实验利用 URL 类和输入输出流实现网络资源下载功能。程序接收用户输入的 URL 和保存路径, 通过 URL.openStream() 获取网络输入流, 将远程资源以字节流形式读取, 并通过 FileOutputStream 写入本地文件。

```

1  import java.io.InputStream;
2  import java.io.FileOutputStream;
3  import java.net.URL;
4  import java.util.Scanner;
5
6  public class FileDownloader {
7
8      public static void download(String fileURL, String savePath) { 1 个用法
9          try {
10              URL url = new URL(fileURL);
11              InputStream in = url.openStream();
12              FileOutputStream out = new FileOutputStream(savePath);
13
14              byte[] buffer = new byte[8192];
15              int len;
16
17              while ((len = in.read(buffer)) != -1) {
18                  out.write(buffer, 0, len);
19              }
20
21              in.close();
22              out.close();
23
24              System.out.println("下载完成! ");
25          } catch (Exception e) {
26              System.out.println("下载失败: " + e.getMessage());
27          }
28      }
29
30  public static void main(String[] args) {
31      Scanner scanner = new Scanner(System.in);
32
33      System.out.print("请输入下载URL: ");
34      String url = scanner.nextLine();
35
36      System.out.print("请输入保存路径 (如 test.html 或 image.png): ");
37      String path = scanner.nextLine();
38
39      download(url, path);
40  }
41  }

```

运行结果示例:

下载 html 文件:

请输入下载URL: <https://www.baidu.com>

请输入保存路径 (如 test.html 或 image.png): [baidu.html](#)

下载完成!

Main.java DNSResolver.java FileDownloader.java <> baidu.html x ResumeDownloader.java

```

1  <!DOCTYPE html>
2  <!--STATUS OK--><html> <head><meta http-equiv=content-type content=text/html;charset=utf-8><meta http-equiv=X-UA
3  </script> <a href=//www.baidu.com/more/ name=tj_briicon class=bri style="...">更多产品</a> </div>

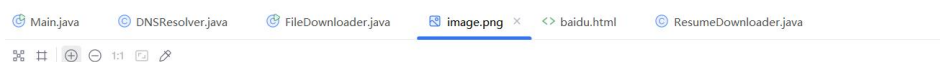
```

下载图片示例:

请输入下载URL: <https://www.python.org/static/img/python-logo.png>

请输入保存路径 (如 test.html 或 image.png): **image.png**

下载完成!



由于采用二进制流读写方式, 程序能够下载 HTML、图片等多种类型的文件, 实现了简单通用的下载功能。在实现过程中, 重点掌握了网络 IO 流与文件 IO 流的结合使用。

3.按内容要求 3 的程序源码及运行结果, **简述思路或实验过程。**

简述思路

在实验二的基础上, 对下载程序进行扩展, 实现断点续传功能。程序首先检测本地是否已存在目标文件, 如果存在则通过 `File.length()` 获取已下载的字节数, 作为断点位置。

随后在 HTTP 请求中设置 Range 请求头 (如 `bytes=已下载字节数-`), 向服务器请求剩余部分数据。同时使用 `RandomAccessFile` 将文件指针移动到断点位置, 通过追加写入方式继续下载, 从而实现断点续传。

此外, 通过注册 `ShutdownHook`, 在程序被中断时能够输出当前已下载文件大小, 实现断点信息的可视化。

实验代码:

```

1  import java.io.*;
2  import java.net.HttpURLConnection;
3  import java.net.URL;
4  import java.util.Scanner;
5
6  public class ResumeDownloader {
7
8      static String savePath; 3个用法
9      static boolean finished = false; 2个用法
10
11     public static void download(String fileURL, String path) { 1个用法
12         File file = new File(path);
13
14         try {
15             long downloaded = file.exists() ? file.length() : 0;
16             System.out.println("当前断点位置: " + downloaded + " 字节");
17
18             URL url = new URL(fileURL);
19             HttpURLConnection conn = (HttpURLConnection) url.openConnection();
20
21             conn.setRequestProperty("Range", "bytes=" + downloaded + "-");
22
23             InputStream in = conn.getInputStream();
24             RandomAccessFile raf = new RandomAccessFile(file, "rw");
25
26             raf.seek(downloaded);
27
28             byte[] buffer = new byte[8192];
29             int len;
30
31             while ((len = in.read(buffer)) != -1) {
32                 raf.write(buffer, 0, len);
33                 System.out.print("\r已下载: " + file.length() + " 字节");
34             }
35
36             raf.close();
37             in.close();
38
39             finished = true;
40
41             System.out.println("\n下载完成!");
42             System.out.println("文件大小: " + file.length() + " 字节");
43
44         } catch (Exception e) {
45             System.out.println("\n下载异常: " + e.getMessage());
46             if (file.exists()) {
47                 System.out.println("当前已下载大小: " + file.length() + " 字节");
48             }
49         }
50     }

```

```

52  public static void main(String[] args) {
53
54      Runtime.getRuntime().addShutdownHook(new Thread(() -> {
55          if (!finished && savePath != null) {
56              File file = new File(savePath);
57              if (file.exists()) {
58                  System.out.println("\n程序被手动中断!");
59                  System.out.println("当前已下载大小: " + file.length() + " 字节");
60                  System.out.println("下次运行将从该位置继续下载");
61              }
62          }
63      }));
64
65      Scanner scanner = new Scanner(System.in);
66
67      System.out.print("请输入大文件URL: ");
68      String url = scanner.nextLine();
69
70      System.out.print("请输入保存路径: ");
71      String path = scanner.nextLine();
72
73      savePath = path;
74
75      download(url, path);
76  }
77

```

测试结果:

第一次下载并且中断:

```

请输入大文件URL: https://dldir1v6.qq.com/weixin/Universal/Windows/WeChatWin\_4.1.8.exe
请输入保存路径 (如 test.zip): wechat.exe
当前断点位置: 0 字节
已下载: 119995425 字节
程序被手动中断!
当前已下载大小: 119995425 字节
下次运行将从该位置继续下载
已下载: 120003617 字节
进程已结束, 退出代码为 130

```

第二次继续下载并完成下载:

```

请输入大文件URL: https://dldir1v6.qq.com/weixin/Universal/Windows/WeChatWin\_4.1.8.exe
请输入保存路径 (如 test.zip): wechat.exe
当前断点位置: 120011809 字节
已下载: 233506664 字节
下载完成!
文件大小: 233506664 字节

进程已结束, 退出代码为 0

```

三、实验总结

（此处写你的过程，比如遇到的错误，以及解决方法，你的所想、所得）

本次实验通过三个部分逐步深入，系统掌握了 Java 网络编程与文件 IO 的基本方法。首先，通过域名解析实验理解了 DNS 的工作原理以及域名与 IP 地址之间的映射关系；其次，通过文件下载实验掌握了 URL 类的使用以及网络数据的获取与本地存储过程；最后，通过断点续传实验深入理解了 HTTP 协议中 Range 请求头的作用，以及如何结合文件随机访问实现下载的续传功能。

在实验过程中，遇到了一些实际问题，例如域名解析结果受到 DNS 环境影响、部分服务器不支持断点续传，以及在 IDEA 中使用停止按钮无法触发中断处理等。通过查阅资料和调试，最终通过更换 DNS、选择支持 Range 的下载资源以及使用终端运行程序并结合 ShutdownHook 成功解决了这些问题。

通过本次实验，不仅提升了对 Java 网络编程的理解，还增强了分析和解决实际问题的能力，对 HTTP 协议和文件操作机制有了更加深入的认识。

AI 使用说明

本人在完成本实验报告过程中，使用了大语言模型作为辅助工具。

使用工具名称：___ChatGPT___（如 ChatGPT / DeepSeek / Qwen 等）

使用用途：

- 查阅资料
- 代码检查
- 报告语言优化
- 其他...

本人承诺：

实验报告中的代码与内容均已理解，并能够向指导老师解释程序实现原理。

学生姓名：___姚泽棉___

学号：___2024150026___

日期：___2026/05/14___

指导教师批阅意见：

成绩评定：

指导教师签字：

年 月 日

备注：

注：1、报告内的项目或内容设置，可根据实际情况加以调整和补充。

2、教师批改学生实验报告时间应在学生提交实验报告时间后 10 日内。